

Abstract Data Types and Concrete Data Types

What Are Abstract and Concrete Data Types?

Abstract data types and concrete data types describe two different layers of thinking about data.

- Abstract Data Type (ADT): what values and operations exist, and what behavior they guarantee.
- Concrete data type (data structure): how those values and operations are represented and executed in memory.

Short version: - ADT = contract - Concrete type = implementation

Example: - Stack ADT defines operations such as PUSH, POP, PEEK, and IS-EMPTY. - A stack can be implemented concretely with either a dynamic array or a linked list.

Formal View

An ADT specifies: - a set of values - a set of operations - semantic rules for operations (preconditions, postconditions, invariants)

An ADT intentionally hides representation details.

A concrete data type specifies: - memory representation - algorithms used to implement each operation - resulting time and space complexity

So one ADT can have multiple concrete implementations with different trade-offs.

Why the Distinction Matters

- It separates specification from implementation.
- It makes code and reasoning modular: users depend on behavior, not internals.
- It allows implementation changes without changing external use.
- It makes analysis clearer:
 - ADT says which operation is needed.
 - Concrete type determines how expensive that operation is.

Common Examples

- List ADT:
 - concrete implementations: dynamic array, linked list
- Queue ADT:
 - concrete implementations: circular array, linked list with head and tail pointers
- Map ADT:

- concrete implementations: hash table, balanced binary search tree

Complexity Perspective

The same ADT can have different performance depending on implementation.

Example: Stack ADT

Concrete implementation	PUSH	POP
Dynamic array	amortized $O(1)$	amortized $O(1)$
Linked list	worst-case $O(1)$	worst-case $O(1)$

Both satisfy the ADT contract, but with different practical behavior and constants.

Terminology

Implicit vs Explicit Data Structures

- Implicit data structure:
 - uses the arrangement of elements to encode structure, rather than storing full pointer links explicitly
 - typically array-based
 - example: binary [[Heaps|heap]] in an array, where parent/child relations are derived from indices
- Explicit data structure:
 - stores structure directly using references or pointers
 - typically node-based
 - example: linked lists and pointer-based trees

Interface, Representation, and Implementation

- Interface: operation names and required behavior (the ADT layer)
- Representation: how data is laid out in memory (array, linked nodes, etc.)
- Implementation: concrete algorithms that realize each operation

Logical Structure vs Physical Layout

- Logical structure: conceptual relationships (for example, parent-child in a tree)
- Physical layout: actual memory placement (contiguous array cells or scattered nodes)

Worst-Case, Amortized, and Average-Case

- Worst-case: upper bound for one operation in the most difficult input state

- Amortized: average per operation over a sequence of operations
- Average-case: expected cost under an input distribution

Common Pitfalls

- Treating an ADT as if it were one specific implementation.
- Comparing algorithms without stating the underlying data structure.
- Ignoring representation constraints (for example, arrays support efficient indexing, linked structures support efficient local insertion/deletion).

Related

- [[Amortized Analysis]]
- [[Priority Queues]]
- [[Searching Algorithms]]
- [[Sorting Algorithms]]
- [[Lectures/Week 4]]